

DSA STUDY BLUEPRINT SERIES

59. Reverse Nodes in K Group Linked List

Reversing Linked List Nodes in K-Groups

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Reverse segments of K nodes. If remaining elements < K, keep unchanged.
- Verify count first; reverse K nodes recursively.

Memory & Execution Blueprint

$[1 \rightarrow 2] (K=2) \cdot [3 \rightarrow 4] \Rightarrow [2 \rightarrow 1] \rightarrow [4 \rightarrow 3]$

C++ Implementation Reference

Standard code layout and syntax

```
Node* reverseKGroup(Node* head, int k) {  
    Node* temp = head;  
    for (int i = 0; i < k; i++) {  
        if (!temp) return head;  
        temp = temp->next;  
    }  
    Node *prev = nullptr, *curr = head, *nxt = nullptr;  
    for (int i = 0; i < k; i++) {  
        nxt = curr->next; curr->next = prev; prev = curr; curr = nxt;  
    }  
    head->next = reverseKGroup(curr, k);  
    return prev;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(N/K)$ recursion stack

Key Practices & Gotchas

- **Important:** The list head node becomes the tail node of the reversed segment.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment 10 → 20:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	curr->next = prev	10 points to nullptr
2	[10, 20, nullptr]	curr->next = prev	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.